

CONTENTS

Interrupciones	
Guía detallada desde cero — con los detalles que salvan el parcial	
0. Por qué existe este problema	
1. Polling vs. Interrupciones — costos	
2. Pedido de interrupción: por nivel vs. por flanco	
3. Atención de un pedido de interrupción — secuencia completa	
4. Identificación del controlador que generó la interrupción	
4.1 Identificación por hardware	
4.2 Identificación por software	
5. Habilitación de interrupciones	
5.1 Enmascaramiento global	
5.2 NMI — interrupción no enmascarable	
5.3 Deshabilitación individual por dispositivo	
6. Prioridades y anidamiento	
6.1 Interrupciones simultáneas	
6.2 “Interrupción de interrupción” — anidamiento	
7. Preservación de contexto: máquina dedicada vs. no dedicada	
7.1 Pseudocódigo verificado — rutina de interrupción en 8086	
8. Diseño completo de un sistema con interrupciones — ejemplo end-to-end	

9. El vector de interrupciones en 8086 — cálculo verificado

10. Tabla resumen general

Ejercicios de repaso rápido (con respuesta)

Resumen ultra-rápido (2 minutos)

Interrupciones

GUÍA DETALLADA DESDE CERO — CON LOS DETALLES QUE SALVAN EL PARCIAL

Arquitectura de Computadoras — FING UdelaR

Polling vs. interrupciones · Nivel vs. flanco · Atención del pedido · Identificación del controlador (hardware y software) · Enmascaramiento y NMI · Prioridades y anidamiento · Contexto en máquina dedicada/no dedicada · 8086: vector de interrupciones, PUSH/POP/IRET

Cómo usar esta guía: cada tema tiene definición corta, procedimiento paso a paso, ejemplos resueltos con verificación explícita, tabla resumen y un recuadro rojo de error frecuente. Este tema apareció en **13 de 13** exámenes analizados, casi siempre combinado con E/S — mereces conocerlo a un nivel de tolerancia cero. Los recuadros amarillos son ejemplos, los verdes son trucos de verificación, y los rojos son errores frecuentes.

0. Por qué existe este problema

Un dispositivo de E/S (teclado, sensor, disco) trabaja a una velocidad muchísimo menor que la CPU. Si la CPU necesita saber cuándo el dispositivo tiene un dato listo, tiene dos estrategias posibles:

Opción 1 — Polling (consulta activa): la CPU ejecuta en loop algo como `while (in(ESTADO) & 0x80) == 0);` — pregunta una y otra vez “¿ya está el dato?” hasta que la respuesta sea sí. El problema: mientras espera, la CPU **no puede hacer nada más**. Si el dispositivo tarda 10 ms en tener un dato y la CPU ejecuta una instrucción cada nanosegundo, hay del orden de millones de ciclos de reloj completamente desperdiciados en preguntar “¿todavía no?” una y otra vez.

Opción 2 — Interrupciones: la CPU le dice al dispositivo “avisame vos cuando tengas algo” y sigue ejecutando otro programa útil (o entra en un estado de bajo consumo). Cuando el dispositivo efectivamente tiene el dato, genera una señal eléctrica (un *pedido de interrupción*) que hace que la CPU, de forma automática y por hardware, deje lo que está haciendo y atienda al dispositivo.

La pregunta que resume el tema: polling es la CPU preguntando “¿ya, ya, ya?” sin parar; interrupción es el dispositivo golpeando la puerta cuando de verdad tiene algo que decir. El costo de la interrupción es la sobrecarga de guardar/restaurar contexto en cada aviso; el

costo del polling es todo el tiempo de CPU perdido entre avisos. Cuando los eventos son poco frecuentes o impredecibles, interrupciones ganan siempre.

1. Polling vs. Interrupciones — costos

Aspecto	Polling	Interrupciones
¿Quién pregunta?	La CPU, activamente, en loop	El dispositivo avisa cuando corresponde
Uso de CPU mientras se espera	100% desperdiciado en el loop de espera	Libre para ejecutar otro programa
Latencia de reacción	Depende de cuán seguido se pregunta	Prácticamente inmediata (hardware)
Complejidad de implementación	Baja — no requiere hardware adicional	Mayor — requiere controlador de interrupciones, vector de rutinas, guardado de contexto
Cuándo conviene	Dispositivo rápido y siempre disponible, o cuando no existe soporte de interrupción en el controlador	Eventos esporádicos/impredecibles, o cuando hay que hacer otro trabajo mientras se espera

Ejemplo mental: Un timer que interrumpe con frecuencia de 10 Hz (Práctico 9, Ejercicio 7) genera una interrupción cada 100 ms. Si en cambio se implementara por polling, la CPU debería revisar el sensor de gas en loop cerrado, sin poder hacer nada más — y como el procesador está dedicado, en este caso concreto el costo de CPU no sería tan grave, pero en un sistema no dedicado (Práctico 10, Ejercicio 2) sería inaceptable: el resto de los programas se congelarían mientras se espera el próximo carácter.

Error frecuente: decir que “las interrupciones no cuestan nada”. Sí tienen costo (guardar y restaurar contexto, identificar el dispositivo, saltar a la rutina), pero ese costo se paga solo cuando ocurre el evento — no en cada ciclo de reloj como el polling.

2. Pedido de interrupción: por nivel vs. por flanco

Definición: el controlador de E/S levanta una señal de salida (habitualmente **INT**) a 1 cuando quiere pedir atención. La CPU tiene una entrada equivalente. Hay dos formas de que la CPU reconozca ese pedido:

- **Por nivel:** la CPU considera que hay un pedido **mientras** la señal INT esté en 1. Se sigue viendo el pedido en cada ciclo hasta que algo la baje.
- **Por flanco:** la CPU considera que hay un pedido solo en el instante en que la señal pasa de 0 a 1 (flanco ascendente). Después de ese instante, aunque la señal siga en 1, no se genera un pedido nuevo.

¿Por qué importa la diferencia? Porque cambia qué pasa si la señal queda activa más tiempo del esperado:

Situación	Por nivel	Por flanco
La rutina de atención tarda en “bajar” la señal (limpiar el registro de estado del dispositivo)	Mientras la CPU no la haya limpiado, la señal sigue en 1 → si se rehabilitan las interrupciones antes de limpiarla, se dispara otra vez el mismo pedido (riesgo de bucle de reinterrupción)	No hay riesgo de repetición: el pedido ya “se consumió” en el flanco, aunque la señal quede en 1 un buen rato
Dos eventos ocurren muy rápido, uno detrás del otro, mientras las interrupciones están deshabilitadas	Se detecta un solo pedido “sostenido” — no importa cuántos eventos se acumularon, alcanza con que el nivel siga en 1	Si el segundo evento ocurre mientras la señal del primero seguía en 1 (no bajó a tiempo), el segundo flanco puede perderse — puede no producirse un segundo flanco ascendente detectable
Ruido eléctrico / rebote	Más tolerante mientras el nivel se mantenga estable	Más sensible: un rebote puede generar flancos espurios

Regla práctica de examen: si el enunciado dice “el hardware coloca 1 en el bit y la CPU debe encargarse de bajarlo al atender”, asumí atención por nivel — la responsabilidad de “apagar” la señal recae en la rutina. Si en cambio el enunciado dice “se invoca la rutina cada vez que aparece un flanco ascendente” (como PULSO en Práctico 10, Ejercicio 5), es por flanco.

Error frecuente: olvidar que en atención por nivel, si la rutina de interrupción no realiza ninguna acción sobre el registro del dispositivo que “apague” el pedido (por ejemplo, leer el dato disponible), al volver a habilitar las interrupciones (`enable()`/`STI`) la CPU va a ver la señal todavía en 1 y va a volver a interrumpir inmediatamente — esto se conoce como quedar “atrapado” en la misma interrupción.

3. Atención de un pedido de interrupción — secuencia completa

Cuando la CPU tiene las interrupciones habilitadas y detecta un pedido (por nivel o flanco, según el hardware), ejecuta esta secuencia **fija**, en este orden:

1. **Termina de ejecutar la instrucción actual.** Una interrupción nunca corta una instrucción a la mitad; se espera a que termine (esto es válido incluso para instrucciones largas como una división).
2. **Guarda el contexto mínimo** necesario para poder volver: como mínimo, la dirección de la próxima instrucción a ejecutar (PC/IP) y el registro de flags. En 8086 esto lo hace automáticamente el hardware con `PUSHF`, `PUSH CS`, `PUSH IP`.
3. **Identifica qué controlador de E/S generó el pedido** (ver sección 4 — por hardware o por software, según la arquitectura).
4. **Obtiene la dirección de la rutina de atención** correspondiente, típicamente consultando una tabla llamada **vector de interrupciones**.
5. **Enmascara las interrupciones** (pone en 0 el flag IF/IF) para que la propia rutina no sea interrumpida por defecto mientras arranca — esto evita, por ejemplo, que una segunda interrupción “pise” el contexto que se está por guardar.
6. **Salta a la rutina de atención** correspondiente y esta ejecuta su trabajo (leer el dato, actualizar variables de estado, actuar sobre el hardware, etc.). Si la rutina necesita usar registros de la CPU, primero debe guardar su valor anterior (ver sección 7).
7. **Restaura el contexto** guardado en el paso 2 (y cualquier registro que la rutina haya usado) y **retorna** (`IRET` en 8086) a la instrucción exactamente siguiente a la que se había interrumpido, con las interrupciones nuevamente habilitadas.

En 8086 esto se ve reflejado directamente en la cartilla de instrucciones: `INT n` hace `PUSHF`, `PUSH CS`, `PUSH IP`, `CS:IP = Mem[n×4]` y pone `I=0`, `T=0`; `IRET` hace `POP IP`, `POP CS`, `POPF` — exactamente los pasos 2, 4-5 y 7 de la lista, hechos por hardware.

Error frecuente de examen: escribir que la CPU “interrumpe la instrucción en curso”. Es falso — la interrupción se atiende recién cuando la instrucción actual termina de ejecutarse

por completo. Lo que se interrumpe es la *secuencia* de instrucciones, no una instrucción individual.

4. Identificación del controlador que generó la interrupción

En cualquier sistema real hay múltiples dispositivos que pueden pedir interrupción. La CPU necesita saber **cuál** de ellos fue, para saltar a la rutina correcta. Hay dos familias de solución — este punto aparece en casi todos los exámenes.

4.1 IDENTIFICACIÓN POR HARDWARE

- **INT/IRQ (líneas dedicadas):** la CPU tiene varias entradas de interrupción físicas (IRQ0, IRQ1, IRQ2...), y cada dispositivo se conecta a una línea distinta. La identificación es directa: la CPU sabe qué línea se activó, y por lo tanto sabe qué dispositivo fue, sin más trabajo. Limitación: la cantidad de dispositivos está acotada por la cantidad de líneas físicas disponibles.
- **INT/INTA con controlador de interrupciones:** cuando hay muchos más dispositivos que líneas físicas, se usa una única línea de pedido **INT** hacia la CPU (conectada en OR cableado desde un **controlador de interrupciones** dedicado, al cual sí se conectan todos los dispositivos por sus líneas **IRQ** individuales). El protocolo es:
 1. Un dispositivo activa su línea IRQ hacia el controlador de interrupciones.
 2. El controlador de interrupciones activa la línea **INT** hacia la CPU.
 3. La CPU, cuando acepta el pedido, responde con **INTA** (interrupt acknowledge).
 4. Al recibir INTA, el controlador de interrupciones coloca en el bus de datos el **identificador** (número de interrupción) asociado a la línea IRQ por la que llegó el pedido.
 5. La CPU usa ese identificador para indexar el vector de interrupciones y saltar a la rutina correcta.
- **Daisy Chain:** es una variante del esquema INT/INTA donde, en lugar de que el controlador de interrupciones resuelva todo, la señal INTA se propaga en cadena, dispositivo por dispositivo, en un orden físico fijo. El primer dispositivo de la cadena recibe el INTA; si él fue quien pidió la interrupción, se “queda” con el INTA (no lo deja pasar) y coloca su identificador en el bus; si no fue él, deja pasar el INTA al siguiente dispositivo de la cadena, y así sucesivamente.

¿Hay prioridad entre dispositivos con Daisy Chain? Sí, y es una prioridad **fija determinada por la posición física en la cadena**: el dispositivo más cercano a la CPU (el primero en recibir INTA) tiene la mayor prioridad, porque siempre tiene la oportunidad de “interceptar” el INTA antes que los demás, sin importar el orden en que hayan pedido la interrupción.

4.2 IDENTIFICACIÓN POR SOFTWARE

Cuando varios controladores de E/S están conectados por **OR cableado a la misma línea INT** (sin un controlador de interrupciones dedicado que identifique el origen), la CPU no tiene forma de saber por hardware cuál fue. La solución es que la propia rutina de atención **consulte, uno por uno, los registros de estado de cada dispositivo candidato**, en un orden fijo, hasta encontrar el que tiene el bit de pedido activo.

Ejemplo (Práctico 9, Ejercicio 4-A): tres dispositivos comparten la misma línea de interrupción. La rutina hace, en alto nivel:

```
void interrupciones() {
    disable();
    if (in(ESTADO_1) & 0x01 != 0)
        atencionDispositivo1();
    else if (in(ESTADO_2) & 0x01 != 0)
        atencionDispositivo2();
    else
        atencionDispositivo3();
    enable();
}
```

El orden de los `if` es la prioridad implícita: si el dispositivo 1 y el 2 piden al mismo tiempo, siempre se atiende primero al 1, porque es el primero que se consulta.

Cómo distinguir en un enunciado cuál mecanismo usar: si el enunciado da **una única línea de interrupción compartida** por varios dispositivos y **varios registros de estado de solo lectura**, uno por dispositivo (como ESTADO_1, ESTADO_2, ESTADO_3), es identificación por software: hay que consultarlos todos dentro de la rutina. Si en cambio el enunciado ya dice “se invoca la rutina X()” de forma específica para cada dispositivo (como `intEntrada()` e `intSalida()` en el Ejercicio 5), la identificación ya la resolvió el hardware — no hace falta ningún polling de registros de estado dentro de la rutina.

Error frecuente: mezclar los dos mecanismos en la misma rutina — por ejemplo, escribir un `if` que compara contra `ESTADO_1` cuando el enunciado ya te dio una rutina específica por dispositivo (`atencionDispositivoN()` ya identificado). Si el hardware ya resolvió la identificación, agregar un polling de estado es trabajo innecesario y en el examen se penaliza como una lectura incorrecta del enunciado.

5. Habilitación de interrupciones

5.1 ENMASCARAMIENTO GLOBAL

La CPU tiene un único bit — llamado según la arquitectura **IF**, **EI**, **IE** o **ET** — que determina si acepta o ignora **todos** los pedidos de interrupción enmascarables, sin importar de qué dispositivo vengan.

- Si el bit está en 1: las interrupciones están habilitadas.
- Si el bit está en 0: ningún pedido de interrupción (enmascarable) es visible para la CPU, aunque la señal INT esté activa.

En 8086, esto se controla con dos instrucciones:

- **STI** (*set interrupt flag*): habilita interrupciones (IF=1).
- **CLI** (*clear interrupt flag*): deshabilita interrupciones (IF=0).

En pseudocódigo de alto nivel (como en los prácticos) esto se expresa como `enable()` / `disable()`.

Automáticamente, cuando la CPU entra a atender una interrupción, pone este bit en 0 (para evitar que la propia rutina sea interrumpida sin control). Por eso, si una rutina quiere permitir anidamiento, debe volver a habilitarlas explícitamente *dentro* de sí misma (ver sección 6).

5.2 NMI — INTERRUPCIÓN NO ENMASCARABLE

Existe una entrada especial, **NMI** (*Non-Maskable Interrupt*), cuyo pedido **siempre se procesa**, sin importar el valor del bit de enmascaramiento global. Se reserva para eventos verdaderamente críticos que no pueden esperar ni ser ignorados — por ejemplo, un fallo inminente de alimentación eléctrica o un error grave de paridad de memoria.

Pregunta típica de examen: “¿por qué existe la NMI si ya existe el bit de enmascaramiento?” — Porque hay eventos donde ignorarlos, aunque sea momentáneamente, puede ser catastrófico (pérdida de datos, daño de hardware). El diseño del sistema decide qué eventos ameritan pasar por esta vía en lugar de la vía enmascarable normal.

5.3 DESHABILITACIÓN INDIVIDUAL POR DISPOSITIVO

Además del enmascaramiento global (todo o nada), cada controlador de E/S suele tener su **propio bit de máscara** en un registro de control, que permite habilitar o deshabilitar **ese dispositivo en particular** sin afectar a los demás. La relación típica es:

salida INT (del dispositivo) = bit de pedido (registro ESTADO) AND bit de máscara (registro CONTROL)

Esto permite, por ejemplo, que un sistema tenga el timer habilitado para interrumpir pero el teclado deshabilitado temporalmente, sin tener que apagar todas las interrupciones globalmente.

Ejemplo: si el registro de control de un dispositivo tiene el bit `MASK_INT` en 0, aunque el dispositivo detecte la condición y ponga en 1 su bit de estado (`INT` interno), la salida efectiva hacia la CPU se mantiene en 0 — ese dispositivo específico queda “silenciado” sin tocar el flag global IF.

Error frecuente: confundir “deshabilitar interrupciones” (global, con `disable()` / `CLI`, afecta a todos los dispositivos) con “enmascarar un dispositivo” (individual, se hace escribiendo un bit en el registro de control de ESE controlador específico). Son dos mecanismos independientes y un examen puede pedir explícitamente cuál corresponde usar.

6. Prioridades y anidamiento

Hay dos escenarios distintos donde el sistema debe decidir qué hacer:

6.1 INTERRUPCIONES SIMULTÁNEAS

Si dos o más dispositivos piden interrupción exactamente al mismo tiempo, se necesita un esquema de prioridades para decidir a cuál atender primero:

- **Prioridad fija:** el orden de atención está predeterminado por diseño (por ejemplo, por la posición física en un Daisy Chain, o por el orden de los `if` en una rutina de identificación por software).
- **Prioridad variable/configurable:** existe una función o registro que permite consultar/cambiar la prioridad de cada dispositivo en tiempo de ejecución (como la función `pri(dispositivo)` del Práctico 9, Ejercicio 4-B).
- **Sin prioridad:** hace falta definir explícitamente algún criterio de desempate (por ejemplo, orden de llegada).

Ejemplo — prioridad fija (Práctico 9, Ej. 4-A): con `pri(displ) > pri(displ2) > pri(displ3)`, la rutina simplemente pregunta en ese orden y atiende al primero que tenga el bit de pedido activo — nunca hace falta comparar valores de prioridad porque el orden de los `if` ya lo garantiza.

6.2 “INTERRUPCIÓN DE INTERRUPCIÓN” — ANIDAMIENTO

Este es el caso donde llega un nuevo pedido de interrupción **mientras la CPU ya está ejecutando la rutina de atención de una interrupción anterior**. Por defecto esto no puede pasar, porque el paso 5 de la sección 3 pone `IF=0` automáticamente al entrar a cualquier rutina. Pero si dentro de la rutina se vuelven a habilitar las interrupciones explícitamente (`enable()` / `STI`) antes de terminar, sí es posible que una nueva interrupción “corte” a la que está en curso.

Regla general: solo se acepta anidar una interrupción de **prioridad igual o mayor** a la que ya se está atendiendo. Para esto se necesita llevar registro del nivel de la interrupción actualmente en curso, para poder compararlo contra el de la nueva solicitud.

Cómo pensarlo en una sola frase: sin re-habilitar interrupciones dentro de la rutina, **nunca** hay anidamiento — la rutina en curso siempre termina antes de que se atienda cualquier otro pedido, sin importar su prioridad. El anidamiento es una decisión de diseño explícita, no algo que “simplemente pasa”.

Error frecuente: asumir que una interrupción de mayor prioridad automáticamente interrumpe a una de menor prioridad en curso, sin verificar que dentro de la rutina se hayan vuelto a habilitar las interrupciones. Si la rutina no hace `enable()` / `STI` en algún punto antes de su `IRET` /retorno, el flag `IF` sigue en 0 durante toda su ejecución y ningún anidamiento es posible, sin importar la prioridad relativa.

7. Preservación de contexto: máquina dedicada vs. no dedicada

Toda rutina de interrupción, al usar registros de la CPU para su propio trabajo, corre el riesgo de destruir valores que el programa interrumpido todavía necesitaba. Cómo se resuelve esto depende del tipo de máquina:

- **Máquina no dedicada:** en la misma máquina corren múltiples programas con propósitos distintos (por ejemplo, una PC de uso general). La rutina de interrupción **no puede asumir nada** sobre qué estaba haciendo el programa interrumpido, así que **debe guardar y restaurar cualquier registro que use**, típicamente en la **pila** (con `PUSH` / `POP` en 8086), antes de tocarlos.
- **Máquina dedicada:** el procesador está exclusivamente destinado a una única función (por ejemplo, el sistema de seguridad de gas del Práctico 9, Ej. 7, o el concentrador de comunicaciones del Ej. 5). Como todo el software — programa principal y rutinas — es escrito por el mismo equipo, se pueden establecer de antemano reglas de uso de recursos compartidos (por ejemplo, “la rutina de interrupción nunca usa el registro BX, por lo tanto no hace falta guardarlo”). Esto permite ahorrar instrucciones de guardado/restauración, a cambio de mayor rigidez y menor portabilidad.

Consecuencia práctica: en máquina dedicada, muchas veces se opta por guardar valores que deben persistir entre invocaciones de la rutina (como contadores de segundos) en **variables globales** en memoria en lugar de la pila, ya que la pila se vacía en cada retorno. Esto es justamente la respuesta a la pregunta teórica del Práctico 10 sobre “cómo recordar valores entre dos ejecuciones de la misma rutina”: usando memoria accesible de forma global, fuera del espacio de la pila.

7.1 PSEUDOCÓDIGO VERIFICADO — RUTINA DE INTERRUPCIÓN EN 8086

Una rutina de interrupción típica en 8086, en una máquina **no dedicada**, sigue esta estructura (basada en el patrón usado en `activarBomba` / `desactivarBomba` y `nuevoChar` del Práctico 10):

```

timerISR proc far
    push ax          ; (1) guardo AX
    push bx          ; (2) guardo BX
    push dx          ; (3) guardo DX

    ; ... trabajo de la rutina, usando ax, bx, dx libremente ...

    pop dx           ; (3') restaura DX – el último guardado, el primero restaurado
    pop bx           ; (2') restaura BX
    pop ax           ; (1') restaura AX
    iret             ; POP IP, POP CS, POPF (hardware) – retorna al programa interrumpido

timerISR endp

```

Verificación explícita de la pila (LIFO): orden de PUSH → AX, BX, DX (la pila queda, de tope a fondo: DX, BX, AX). El orden de POP debe ser exactamente el inverso: primero DX (el que quedó arriba de todo), después BX, después AX. Si el POP se hiciera en el mismo orden que el PUSH (ax, bx, dx) los valores quedarían cruzados: AX recibiría lo que era DX, y viceversa — un error silencioso que corrompe el programa interrumpido sin generar ningún mensaje de error.

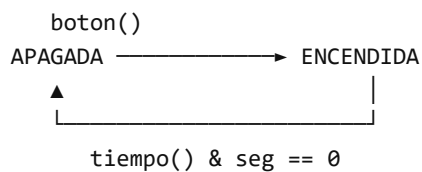
Cada PUSH tiene su POP correspondiente: 3 PUSH (ax, bx, dx) ↔ 3 POP (dx, bx, ax) ✓. El puntero de pila SP queda exactamente donde estaba antes de entrar a la rutina, condición necesaria para que el `IRET` final recupere la dirección de retorno correcta (IP, CS) y los flags que el hardware había apilado automáticamente al entrar.

Error frecuente de tolerancia cero: hacer `push ax / push dx` al entrar y `pop ax / pop dx` (mismo orden) antes de retornar, en lugar de invertir el orden. Esto desbalancea la pila de forma que **no** se nota en la ejecución de la rutina en sí (los PUSH y POP están en igual cantidad), pero **sí** corrompe los valores restaurados en los registros, y en casos más graves, si el error ocurre sobre el manejo de `SP` antes del `IRET`, puede hacer que el `IRET` recupere una dirección de retorno incorrecta.

8. Diseño completo de un sistema con interrupciones — ejemplo end-to-end

Enunciado (basado en Práctico 9, Ej. 6): un sistema de iluminación de pasillos tiene un botón de encendido por piso. Al presionar cualquier botón se debe encender la luz (bit 0 de la dirección `0x20`) y apagarla automáticamente a los 45 segundos si no se vuelve a presionar ningún botón. Se cuenta con una rutina `boton()`, invocada por interrupción al presionar cualquier botón, y una rutina `tiempo()`, invocada por interrupción de un timer cada 1 segundo.

Diseño de la máquina de estados (dos estados: `APAGADA`, `ENCENDIDA`):



Rutinas (alto nivel, verificadas):

```

#define APAGADA 0
#define ENCENDIDA 1
char estado = APAGADA;
int seg;           // contador de segundos restantes hasta apagar

void boton() {
    disable();
    if (estado == APAGADA) {
        estado = ENCENDIDA;
        char byte = in(0x20);
        byte = byte | 0x01;    // enciendo el bit 0
        out(0x20, byte);
    }
    seg = 45;                // (re)inicio la cuenta regresiva, esté prendida o
no
    enable();
}

void tiempo() {
    disable();
    if (estado == ENCENDIDA) {
        seg = seg - 1;
        if (seg == 0) {
            estado = APAGADA;
            char byte = in(0x20);
            byte = byte & 0xFE;    // apago el bit 0, sin tocar los demás bits
            out(0x20, byte);
        }
    }
    enable();
}

```

Verificación de la secuencia completa de atención (aplicando los 7 pasos de la sección 3) ante un evento real: un usuario presiona el botón del piso 2 mientras el programa principal ejecuta cualquier otra tarea.

1. La CPU termina la instrucción en curso del programa principal.
2. Guarda PC/flags (automático, por hardware).
3. Identifica que fue el controlador del botón (línea dedicada o registro de estado, según el diseño físico).
4. Busca en el vector de interrupciones la dirección de `boton()`.
5. Enmascara interrupciones (IF=0) y salta a `boton()`.
6. `boton()` ejecuta: si estaba apagada, prende el bit 0 de 0x20 y pasa a ENCENDIDA; en

cualquier caso, reinicia `seg` a 45.

7. Restaura contexto y retorna (IRET) exactamente a la instrucción del programa principal que había sido interrumpida, con IF=1 de nuevo.

Cada segundo, el mismo mecanismo se dispara para `tiempo()`, que decrementa `seg` y apaga la luz cuando llega a 0 — **solo si** nadie volvió a presionar un botón antes (lo cual habría reiniciado `seg` a 45 desde `boton()`).

Error frecuente en este tipo de ejercicio: usar un contador que cuenta “hacia arriba” (`seg = seg + 1`) y comparar contra 45 en `tiempo()`, pero ****no**** reiniciarlo a 0 dentro de `boton()` — al presionar un segundo botón antes de los 45 segundos, si no se reinicia el contador, las luces se apagan con el tiempo restante del primer botón en lugar de reiniciar la cuenta completa. Cualquiera de las dos convenciones (contar hacia arriba desde 0, o hacia abajo desde 45) es válida, pero `boton()` siempre debe reiniciar el contador.

9. El vector de interrupciones en 8086 — cálculo verificado

En 8086, el **vector de interrupciones** ocupa las direcciones absolutas **0x00000 a 0x003FF** (256 entradas × 4 bytes cada una = 1024 bytes = 0x400). Esta ubicación es fija, determinada por el hardware del procesador — no se puede reubicar.

Cada entrada tiene 4 bytes: los **2 bytes bajos = offset** (desplazamiento) de la rutina, y los **2 bytes altos = segmento**.

Fórmula para encontrar la entrada de la interrupción n:

Dirección de la entrada = $n \times 4$ (offset guardado en esa dirección; segmento guardado 2 bytes más adelante, en $n \times 4 + 2$)

Ejemplo 1 — interrupción 4, rutina en la dirección física 0x41024:

- Descompongo la dirección física en segmento:desplazamiento. Como $DIR = SEGMEN TO \times 16 + DESPLAZAMIENTO$, elijo $SEGMEN TO = 0x4000$, $DESPLAZAMIENTO = 0x1024$ ($0x4000 \times 16 + 0x1024 = 0x40000 + 0x1024 = 0x41024$ ✓).
- Calculo la posición en el vector: $n \times 4 = 4 \times 4 = 16$ (decimal) = **0x10** (hexadecimal).
- Instalo: offset en ES:[0x10], segmento en ES:[0x12].

```
cli
mov ax, 0
mov es, ax
mov word ptr es:[0x10], 0x1024 ; offset
mov word ptr es:[0x12], 0x4000 ; segmento
sti
```

Discrepancia encontrada y corregida — atención, error real hallado en el material de

Práctico 10: la solución oficial de este mismo ejercicio (Práctico 10, Ej. 1-B) escribe `es:[0x16]` y `es:[0x18]` en lugar de `es:[0x10]` y `es:[0x12]`. El error es puntual y muy fácil de cometer: $4 \times 4 = 16$ en **decimal**, pero 16 decimal **no es** “0x16” — convertido a hexadecimal, 16 decimal es **0x10**. El error consiste en tomar el resultado decimal del cálculo (16) y pegarle el prefijo “0x” directamente, en lugar de convertirlo a hexadecimal. Lo mismo ocurre en el Ejercicio 3-C del mismo práctico (interrupción 9): $9 \times 4 = 36$ decimal = **0x24** hexadecimal — no “0x36” como aparece en esa solución. Redobló el cuidado con este paso puntual: **calculá siempre $n \times 4$ en decimal, y recién después convertí ese resultado a hexadecimal** — nunca pegues el “0x” sobre el número decimal.

Truco de verificación rápida: multiplicar por 4 en binario/hex es desplazar 2 bits a la izquierda. Para $n=4$ (0x04): $0x04 \ll 2 = 0x10$ ✓. Para $n=9$ (0x09): $0x09 \ll 2 = 0x24$ ✓. Usar el corrimiento de bits directamente en hexadecimal evita el error de mezclar decimal y hexadecimal a mitad de camino.

Ejemplo 2 — interrupción 13, rutina en la dirección física 0x55900 (Práctico 10, Ej. 4-C):

1. Posición en el vector: $13 \times 4 = 52$ decimal = **0x34** hexadecimal (verificación: $0x0D \ll 2 = 0x34$ ✓).
2. Segmento: desplazamiento para 0x55900 — el enunciado exige elegir un segmento tal que el CS guardado sea el mismo para dos rutinas distintas (13 y 14, en 0x55900 y 0x5600C). Tomando segmento = 0x5000: desplazamiento₁ = $0x55900 - 0x50000 = 0x5900$; para la interrupción 14 en 0x5600C con el mismo segmento 0x5000: desplazamiento₂ = $0x5600C - 0x50000 = 0x600C$. Ambas verifican: $0x5000 \times 16 + 0x5900 = 0x50000 + 0x5900 = 0x55900$ ✓; $0x5000 \times 16 + 0x600C = 0x50000 + 0x600C = 0x5600C$ ✓.

10. Tabla resumen general

Concepto	Idea clave
Polling	La CPU pregunta activamente en loop — desperdicia ciclos, simple de implementar
Interrupción	El dispositivo avisa; la CPU hace otra cosa mientras tanto
Por nivel	Se detecta mientras la señal esté en 1 — riesgo de reinterrupción si no se limpia
Por flanco	Se detecta solo en la transición 0→1 — riesgo de perder un segundo evento muy próximo
Atención (7 pasos)	Termina instrucción → guarda contexto → identifica dispositivo → busca rutina en el vector → enmascara → ejecuta rutina → restaura y retorna (IRET)
Identificación por hardware	INT/IRQ (líneas dedicadas), INT/INTA (+ opcional Daisy Chain con prioridad por posición física)
Identificación por software	La rutina consulta registros de estado en un orden fijo — ese orden define la prioridad implícita
Enmascaramiento global	Un solo bit (IF/EI) habilita o inhibe todos los pedidos enmascarables
NMI	Pedido que siempre se procesa, sin importar el enmascaramiento global
Deshabilitación individual	Bit de máscara propio de cada controlador — no afecta a los demás dispositivos
Prioridad fija/variable	Determina cuál se atiende primero ante pedidos simultáneos
Interrupción de interrupción	Solo ocurre si la rutina en curso re-habilita las interrupciones explícitamente
Máquina dedicada	Un solo propósito — puede ahorrar guardado de registros por convención de diseño
Máquina no dedicada	Múltiples programas — la rutina SIEMPRE debe guardar/restaurar lo que use
Vector de interrupciones (8086)	0x00000–0x003FF; entrada de la interrupción n en la dirección n×4 (offset), n×4+2 (segmento)

Ejercicios de repaso rápido (con respuesta)

Pregunta	Respuesta
¿Por qué el polling es ineficiente?	Porque la CPU queda atrapada consultando en loop y no puede hacer ningún otro trabajo mientras espera el evento
Atención por nivel: ¿qué pasa si la rutina no “limpia” la señal antes de rehabilitar interrupciones?	Se vuelve a detectar el mismo pedido inmediatamente — riesgo de reinterrupción en bucle
Atención por flanco: ¿qué riesgo tiene frente a nivel?	Puede perderse un segundo evento si ocurre mientras las interrupciones siguen deshabilitadas y no genera un flanco detectable nuevo
¿Cuál es el primer paso, sin excepción, al atender una interrupción?	Terminar de ejecutar la instrucción actual — nunca se corta una instrucción a la mitad
Con Daisy Chain, ¿qué determina la prioridad entre dispositivos?	La posición física en la cadena: el más cercano a la CPU intercepta el INTA primero
¿Cuándo se usa identificación por software en lugar de por hardware?	Cuando varios dispositivos comparten una única línea INT sin un controlador de interrupciones dedicado que resuelva el origen
¿Qué hace que un pedido por NMI sea distinto a uno normal?	Siempre se procesa, sin importar el valor del bit de enmascaramiento global
¿Alcanza con que una interrupción tenga mayor prioridad para que anide a una en curso?	No — además la rutina en curso debe haber rehabilitado explícitamente las interrupciones antes de que llegue el nuevo pedido
Máquina no dedicada: ¿puede la rutina de interrupción usar un registro sin guardarlo antes?	No — debe guardarlo (típicamente con PUSH) y restaurarlo (POP) antes de retornar
En 8086, PUSH ax; PUSH bx; PUSH dx al entrar a una rutina — ¿en qué orden van los POP?	dx, bx, ax (orden inverso al PUSH)
¿Dónde está el vector de interrupciones del 8086 y puede reubicarse?	0x00000 a 0x003FF; no, es una posición fija del diseño del hardware

Pregunta	Respuesta
Interrupción número 6: ¿en qué dirección del vector está su entrada?	$6 \times 4 = 24$ decimal = 0x18 hexadecimal (offset en ES: [0x18], segmento en ES:[0x1A])
¿Qué instrucciones 8086 habilitan/deshabilitan interrupciones globalmente?	STI (habilita, IF=1) y CLI (deshabilita, IF=0)
¿Qué hace automáticamente la instrucción INT n en 8086?	PUSHF, PUSH CS, PUSH IP, salta a CS:IP = Mem[n×4], pone IF=0 y TF=0
¿Qué hace automáticamente IRET?	POP IP, POP CS, POPF — retorna y restaura todos los flags, incluyendo IF

Resumen ultra-rápido (2 minutos)

- **Polling** = preguntar en loop, desperdicia CPU. **Interrupción** = el dispositivo avisa, la CPU hace otra cosa mientras tanto.
- **Nivel** = mientras la señal esté en 1 (riesgo: reinterrupción si no se limpia). **Flanco** = solo en la transición 0→1 (riesgo: perder un segundo evento cercano).
- **Atención en 7 pasos:** termina instrucción → guarda PC/flags → identifica el dispositivo → busca la rutina en el vector → enmascara interrupciones → ejecuta la rutina → restaura y retorna (IRET).
- **Identificación:** por hardware (INT/IRQ directo, o INT/INTA + opcional Daisy Chain con prioridad por posición física) o por software (la rutina consulta registros de estado en orden — ese orden es la prioridad).
- **Habilitación:** un bit global (IF/EI, con STI/CLI en 8086) enmascara todo; NMI nunca se enmascara; cada dispositivo además tiene su propio bit de máscara individual.
- **Prioridades:** fija, variable o sin prioridad, ante pedidos simultáneos. El anidamiento (“interrupción de interrupción”) solo ocurre si la rutina en curso rehabilita interrupciones explícitamente, y solo para prioridad igual o mayor.
- **Contexto:** máquina no dedicada → siempre guardar/restaurar registros usados (PUSH al entrar, POP en orden inverso antes de IRET). Máquina dedicada → puede ahorrarse guardados por convención de diseño, y usa variables globales (no la pila) para recordar valores entre invocaciones.
- **8086:** vector en 0x00000–0x003FF, entrada de la interrupción n en la dirección $n \times 4$ (offset) y $n \times 4 + 2$ (segmento) — calculá siempre $n \times 4$ en decimal y convertí recién después a hexadecimal, nunca pegues el “0x” sobre el número decimal.